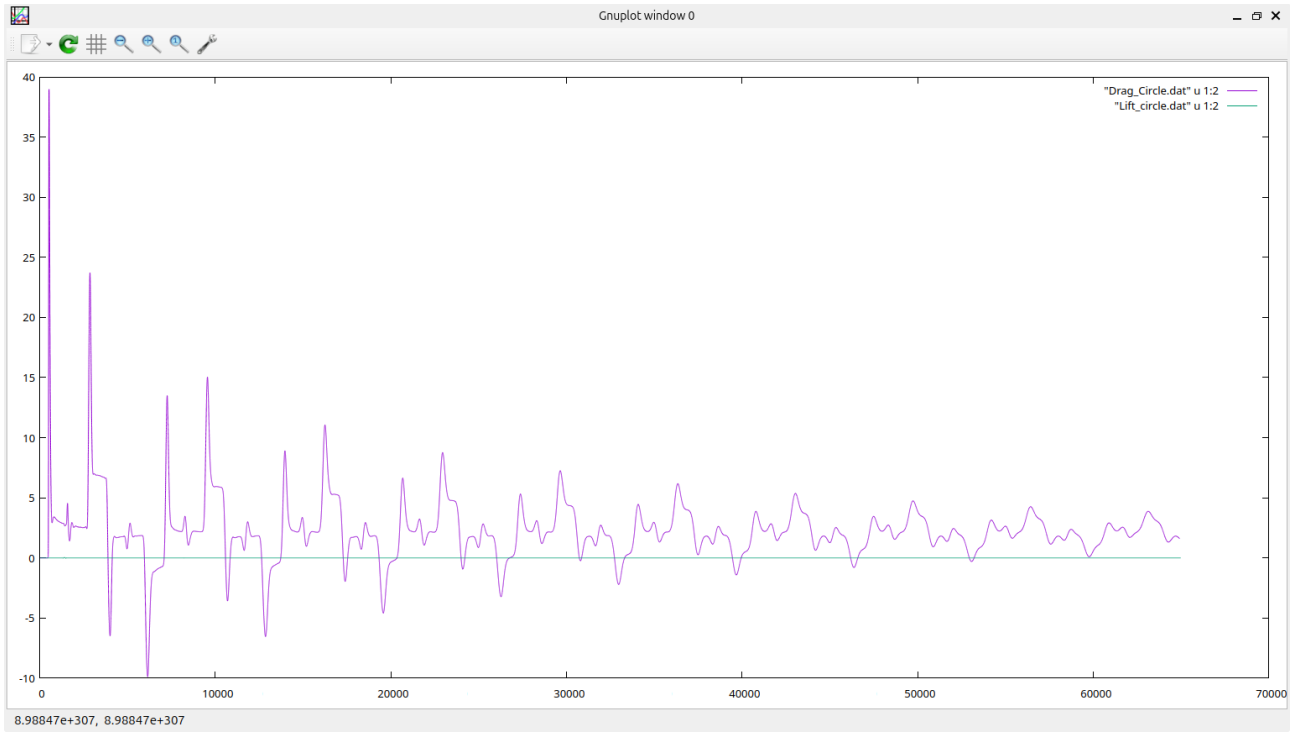


Cold Start Problem

by navaneeth

If you are seeing errors like this in the temporal history of forces like drag and lift:



you can see that there are massive spikes and non decaying waves with this errors that that refuse to settle (even in the steady regimes like $Re = 20$, 40 (the above image is at $Re = 20$))

Then congratulations your solver is working perfectly! What you are simulating is the acoustic wave bouncing across the fluid domain!

WHY THIS HAPPENS!

What is happening here is that when the fluid domain is initialized at a dead stop condition ($u = 0$, $v = 0$) and the inlet boundary condition is set at a high velocity (e.g. $Ma = 0.1$) at $t = 0$, it creates a pressure shock wave (water hammer).

What is water hammer?

A water hammer is just a pressure surge caused when a fluid's momentum is forced to change instantaneously.

WHY DOES THIS NOT DIE DOWN?

A good question to ask now would be this i.e. why does this shock not die out with time? When we throw a rock into a pond the ripples eventually fade but why does it not fade here?

Assuming you are using Zou-He boundary condition and BGK operator the reason is both of them!

A. The role of Zou-He boundary condition:

Zou-He Boundary conditions act like rigid walls which do not absorb the pressure waves but it actually reflects it back into the fluid domain! This causes the wave to not die out at all.

B. The role of BGK operator

we know our BGK solver is a single relaxation time solver! This relaxation time itself is our problem, the thing is when running our code we will have our relaxation time relatively low (above 0.5 but still low) and we know that the fluid's viscosity is dictated by our relaxation time! So this low relaxation time means that the fluid lacks the internal friction required to absorb and kill the shockwave's energy!

Also as a side note, yes we could just increase the number of nodes and increase the relaxation time,

the equation of the relaxation time is

$$\text{Tau} = (3*((U * d)/\text{Re})) + 0.5$$

where, d is a representation of the number of nodes (in in our required need)

so yes an increase in number of nodes means higher Tau

BUT we also do have to keep the computation time in our mind too!!

MATHEMATICAL PROOF!

You could of course not believe me or the reason for your error might be different. So to fix this we can have a mathematical proof,

Check the location of your first spike (in the temporal sense of course, I mean just check at which time your first spike is happening)

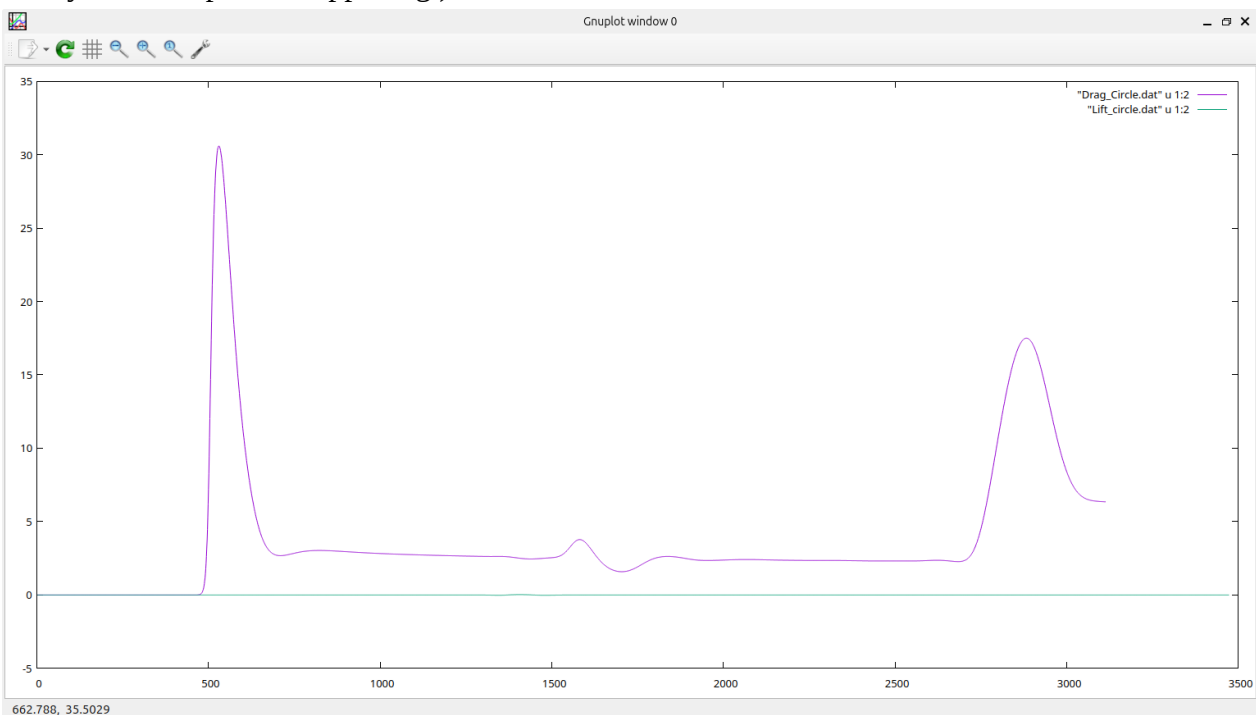


image 2 : zoomed in image

it is => *distance from inlet to cylinder*/ C_s (and you know $C_s = 1 / \sqrt{3}$)

in my case $d = 32$, radius = 16, location of center = $10 * d = 320$

$$\text{distance from inlet to cylinder} = 320 - 16 = 304$$

=> $(320 - 16) / (1 / \sqrt{3}) = \underline{526.54}$; which is exactly where the location of the first peak is at!

And if you want to make sure that this is not a fluke and double verify this we can also predict the location of the second spike

it is => (*Distance from the cylinder to the outlet + outlet back to cylinder*)/ C_s

$$\text{which is} = ((30 * d) + ((30 * d) - ((10 * d) + 16))) / (1 / \sqrt{3})$$

$$= 1584 / (1 / \sqrt{3}) = \underline{2743.56}$$

and again you can see that the second peak also is exactly where we predicted it to be at!

NOTE: if you are using the output values (velocity and density) of the last code as the input value of this code then the temporal location might (most likely will not) align with this method!

HOW DO WE SOLVE THIS PROBLEM

The method that I have used to solve this is just initialize the entire fluid domain to be moving at U_{star} right at the start!

Now this will cause a very huge spike of C_d at time step 1 because the velocity is increasing at infinite gradient but this problem is solved very quickly and we reach the steady state much more quickly

p.s there are actually other method to solve this too, like gradually increasing the inlet velocity but those are just waste of time according to me! In my opinion my method is so much better

```
vector2D vel_init(U_star, 0.0);

for(int i = 0; i < total_nodes; i++)
{
    mesh[i].rho = rho;
    mesh[i].vel = vel_init; // Initialize to U_star
    mesh[i].ratio = 0.0;

    // This initializes the populations f_i to equilibrium based on U_star
    mesh[i].equilibrium(mesh[i].rho, mesh[i].vel, Cs, rho_0);
}
```

how I have done it in code.